

make2DimArray [SimTalk]

Creates a *two-dimensional array* from the values of the specified *one-dimensional array*.

Type

Function

Syntax

```
make2DimArray(xDim:integer, arrayData:any[]) -> any[xDim,*]
```

Parameters

You can specify the following parameters:

- The parameter *xDim* of data type *integer* sets the X-dimension of the *array* to be created. *Plant Simulation* automatically determines the Y-dimension according to the values contained in the *one-dimensional array*.
- The parameter *arrayData* of data type *any[]* contains the values with which the *two-dimensional array* is going to be filled.

Return Value

The return value has the data type *any[]*.

This is a *two-dimensional array* that has the same base data type as the original *one-dimensional array*.

Examples

```
var a : integer[3,2] := make2DimArray(3, [11,21,31, 12,22,32])
```

```
var array1Dim : integer[] := [1, 21, 2, 22, 3, 23]
Connector.CornerPointsArray := make2DimArray(2, array1Dim)
```

Functions for Debugging the Model

_Functions for Debugging the Model

SimTalk provides the functions listed in the table of contents to the left for debugging your simulation models.

clearAllBreakpoints [SimTalk]

Deletes all class breakpoints and all instance breakpoints you have set in all *Methods* in your simulation model.

Type

Function

Syntax

```
clearAllBreakpoints
```

See also

[Delete Breakpoints in All Methods](#)

debug [SimTalk]

Stops the simulation run and opens the *Debugger* window. If your model contains errors you can interrupt the simulation run and debug your model.

Type

Function

Syntax

```
debug
```

Example

```
if determinante 0  
  debug  
end
```

See also

[Debug Method \[button\]](#)

Video on YouTube

<https://youtu.be/PYXYutlaGOs?si=6ZNwfhIGkAQHK4II&t=725>

deleteSuspendedMethods [SimTalk]

Terminates all kinds of suspended *Methods*, meaning *Methods* that were suspended by a *waituntil-statement*, a *stopuntil-statement*, or by a *sleep-instruction*.

Remarks

deleteSuspendedMethods does not delete the *Method* object itself, but only terminates the suspended state of this *Method*.

The *EventController* automatically **terminates all** suspended *Methods* during the *reset phase* of the simulation.

Type

Function

```
deleteSuspendedMethods([WaitingToo:boolean:=false,  
OnlyLocal:boolean:=false])
```

Parameters

You can specify the following parameters:

- The optional parameter *WaitingToo* of data type *boolean* sets if *Methods*, which are suspended by a *wait instruction*, will be deleted as well (*true*) or not (*false*).

Default Value of the Parameter

The default value is *false*.

- The optional parameter *OnlyLocal* of data type *boolean* sets if *Plant Simulation* only deletes local suspensions, which are located in the *root-Frame* or in one of *its sub-Frames*, as well as *Methods*, whose caller is located in the *root-Frame* or in one of *its sub-Frames* (*true*) or not (*false*).

Default Value of the Parameter

The default value is *false*.

Example

```
deleteSuspendedMethods(true, true)
```

See also

[Terminate Suspended Methods](#)

[Suspending Methods](#)

Reset Simulation [EventController]

waituntil [SimTalk] and stopuntil [SimTalk]

sleep [SimTalk]

getErrorStop [SimTalk]

Returns the settings **Stop on Formulas** and **Stop on Error Handlers** and sets the passed variables to the respective values.

Type

Function

Syntax

```
getErrorStop(byRef StopOnErrors:boolean, byRef  
StopOnErrorsInFormulas:boolean) → boolean
```

Parameters

You can specify the following local variables:

- The local variable *StopOnErrors* of data type *boolean* is the variable into which *Plant Simulation* writes if **Stop on Error Handlers** is activated or deactivated.
- The local variable *StopOnErrorsInFormulas* of data type *boolean* is the variable into which *Plant Simulation* writes if **Stop on Formulas** is activated or deactivated.

Return Value

The return value has the data type *boolean*.

Example

```
var StopOnErrors, StopOnErrorsInFormulas: boolean  
getErrorStop(StopOnErrors, StopOnErrorsInFormulas)  
print StopOnErrors, " ", StopOnErrorsInFormulas
```

See also

[setErrorStop \[SimTalk\]](#)

[Stop on Error Handlers](#)

[Stop on Formulas](#)

ignoreBreakpoints [SimTalk]

Ignores (`true`) or does not ignore (`false`) user-defined breakpoints during your simulation run.

Type

Function

Syntax

```
ignoreBreakpoints(Ignore: boolean)
```

Parameter

The parameter *Ignore* of data type *boolean* designates the action to be taken.

Example

```
ignoreBreakpoints(true)
```

See also

[Ignore Breakpoints](#)

setErrorHandler [SimTalk]

Sets the *Method*, whose name you enter as the parameter of data type *object*, as the global error handling method.

Remarks

When you set a global error handling method, *Plant Simulation* will not open the *Method Debugger* when a runtime error occurs in a *SimTalk* method. Instead *Plant Simulation* will terminate the entire call chain (the method that caused the runtime error and all calling methods) and call the global error handling method.

Note that the global error handling method will only be called, if *Plant Simulation* did not find individual **Error Handling** for methods.

When you set `void` as error handling method, *Plant Simulation* deactivates global error handling and opens the *Debugger* when a runtime error occurs and an individual error handler does not exist.

Three parameters are passed to the error handling method:

- A *string* with the error message. You have to declare the *string* parameter as a by reference parameter.
- A *string* with the path to the *Method* with faulty source code.
- An *integer* value that tells, which line in the source code contains the error.

Type

Function

Syntax

```
setErrorHandler(Method:object) → object
```

Parameter

The parameter *Method* of data type *object* designates the *Method*.

Return Value

The return value has the data type *object*.

setErrorHandler returns the last registered error handling method.

Examples

```
// source code of myMethod  
param byref ErrorMessage:string, MethodPath:string, LineNumber:integer  
  
print "Error in ", MethodPath, " line ", LineNumber, ": ", ErrorMessage  
ErrorMessage := "" // catch this error
```

```
setErrorHandler(&myMethod)
```

See also

[Select Template](#) in the Method

[Error Handling](#)

[Calculate Values with a Formula](#)

[getCallStack](#) [SimTalk]

[throwRuntimeError](#) [SimTalk]

setErrorStop [SimTalk]

Activates (`true`) or deactivates (`false`) the features **Stop on Formulas** and **Stop on Error Handlers**.

Type

Function

Syntax

```
setErrorStop(StopAfterError:boolean[, StopAfterErrorInFormula:boolean]) →  
boolean
```

Parameters

You can specify the following parameters:

- When you specify `true` for the parameter *StopAfterError*, *Plant Simulation* stops the simulation after an error occurred in the current method and opens the *Debugger* with the method containing the error and highlights the line containing the error in red. When you close the *Debugger*, *Plant Simulation* terminates the entire call chain. If the events list in the *EventController* contains any more events, the simulation continues.

When you specify `false`, *Plant Simulation* terminates the entire call chain and the simulation continues.

- The optional parameter *StopAfterErrorInFormula* of data type *boolean* determines if *Plant Simulation* stops the simulation, when it encounters errors in formulas (`true`) or if it does not stop it (`false`).

Return Value

The return value has the data type *boolean*.

Example

```
setErrorStop(false, true)
```

See also

[getErrorStop \[SimTalk\]](#)

[Stop on Error Handlers](#)

[Stop on Formulas](#)

setMaxDepthOfCalls [SimTalk]

Sets the number of methods that *Plant Simulation* may call.

Remarks

Each *Method* consumes main memory while being executed. *Plant Simulation* only releases the memory after the execution of a *Method* is finished. This means that memory consumption increases with each *Method* that calls another method before it finishes. The number of *Methods* is the *depth of calls*. For example, when method A calls method B, and method B in turn calls method C, the depth of calls is three. If, by accident, method C called itself again and again, the depth of calls would be infinite. To avoid *Plant Simulation* running out of memory, limit the *depth of calls*. We found the default value 500 to be sufficient for most applications.

Type

Function

Syntax

```
setMaxDepthOfCalls(NumberOfMethods:integer) → integer
```

Parameter

The parameter *NumberOfMethods* of data type *integer* designates the number of methods that *Plant Simulation* may call.

Return Value

The return value has the data type *integer*.

Is the previous value of the maximum number of suspended methods and does not set the new value if 0 is passed.

Example

```
setMaxDepthOfCalls(1500)
```

See also

[Maximum Depth of Calls \[preferences\]](#)

setMaxEdgeLengthWorkerRouteNetwork [SimTalk]

Sets the **Maximum Edge Length**. When computing the **Worker Route Network**, *Plant Simulation* only uses line segments, which are equal to or shorter than the specified value.

Remarks

You can type in a value between 3 meters and 100 meters, which applies in all directions. 0 does not limit the **edge length**. The value applies to the computation of the **route network** of *Workers*, who are walking freely in the area of the simulation model.

Type

Function

Syntax

```
setMaxEdgeLengthWorkerRouteNetwork(newMaxEdgeLength:length) -> length
```

Parameter

The parameter *newMaxEdgeLength* of data type *length* designates the new edge length.

Return Value

The return value has the data type *length*.

Is the previous value of the model setting.

Example

```
var oldMaxLength : length  
oldMaxLength := setMaxEdgeLengthWorkerRouteNetwork(8m)  
-- limits the edge length to 8 meters
```

See also

[Maximum Edge Length \[model settings\]](#)

[Maximum Edge Length \[preferences\]](#)

setMaxNumberOfCallChains [SimTalk]

Limits the **Maximum Number of Call Chains** which *Plant Simulation* calls.

Remarks

When *Plant Simulation* reaches the number of call chains you set, it stops the simulation and shows an error message.

You can then increase the **Maximum Number of Call Chains** and continue the simulation.

Type

Function

Syntax

```
setMaxNumberOfCallChains(NumberOfCallChains:integer) → integer
```

Parameter

The parameter *NumberOfCallChains* of data type *integer* designates the maximum number of call chains.

Return Value

The return value has the data type *integer*.

Is the previous value of the **Maximum Number of Call Chains** and does not set the new value if 0 is passed.

Example

```
setMaxNumberOfCallChains(1500)
```

See also

[Maximum Number of Call Chains \[preferences\]](#)

setMaxNumberOfSamples [SimTalk]

Sets the **Maximum Number of Samples** to prevent *Plant Simulation* from spending an excessive amount of time for die rolls, which are caused by an unfortunate selection of the interval bounds.

Remarks

You can specify a number between 1 and 32000. *Plant Simulation* ignores values outside of this range without issuing a warning message and keeps the previous value. The greater the value you enter, the later you recognize if the bounds are too close together. For a normal distribution it may, for instance, happen that *Plant Simulation* draws extremely large values. When the bounds are too close together, this may result almost exclusively in outliers. Then *Plant Simulation* has to create a large number of random numbers consecutively before a valid value is available. This can take up a lot of time.

Type

Function

Syntax

```
setMaxNumberOfSamples(NumberOfSamples:integer) → integer
```

Parameter

The parameter *NumberOfSamples* of data type *integer* designates the maximum number of die rolls.

Return Value

The return value has the data type *integer*.

Is the previous value of the **Maximum Number of Samples** and does not set the new value if 0 is passed.

Example

```
setMaxNumberOfSamples(12)
```

See also

[Maximum Number of Samples \[preferences\]](#)

setMaxSuspendedMethods [SimTalk]

Sets the maximum number of methods that may be suspended at any one time. The number of suspended methods by *waituntil*-statements is limited.

Remarks

When *Plant Simulation* reaches the number you set, any additional methods will not be suspended and *Plant Simulation* will display an error message.

You can then either increase the **Maximum Number of Suspended Methods** and continue your simulation run or stop the simulation and fix the modeling error.

Type

Function

Syntax

```
setMaxSuspendedMethods(NumberOfSuspendedMethods:integer) → integer
```

Parameter

The parameter *NumberOfSamples* of data type *integer* designates the maximum number of samples.

Return Value

The return value has the data type *integer*.

Is the previous value of the **Maximum Number of Suspended Methods** and does not set the new value if 0 is passed.

Example

```
setMaxSuspendedMethods(40)
```

See also

[Maximum Number of Suspended Methods \[preferences\]](#)